Assignment 21.5

Name: Sushma Purella

Batch:44

H. No:2303A52188

Task A – AI-Based Initial Program Development

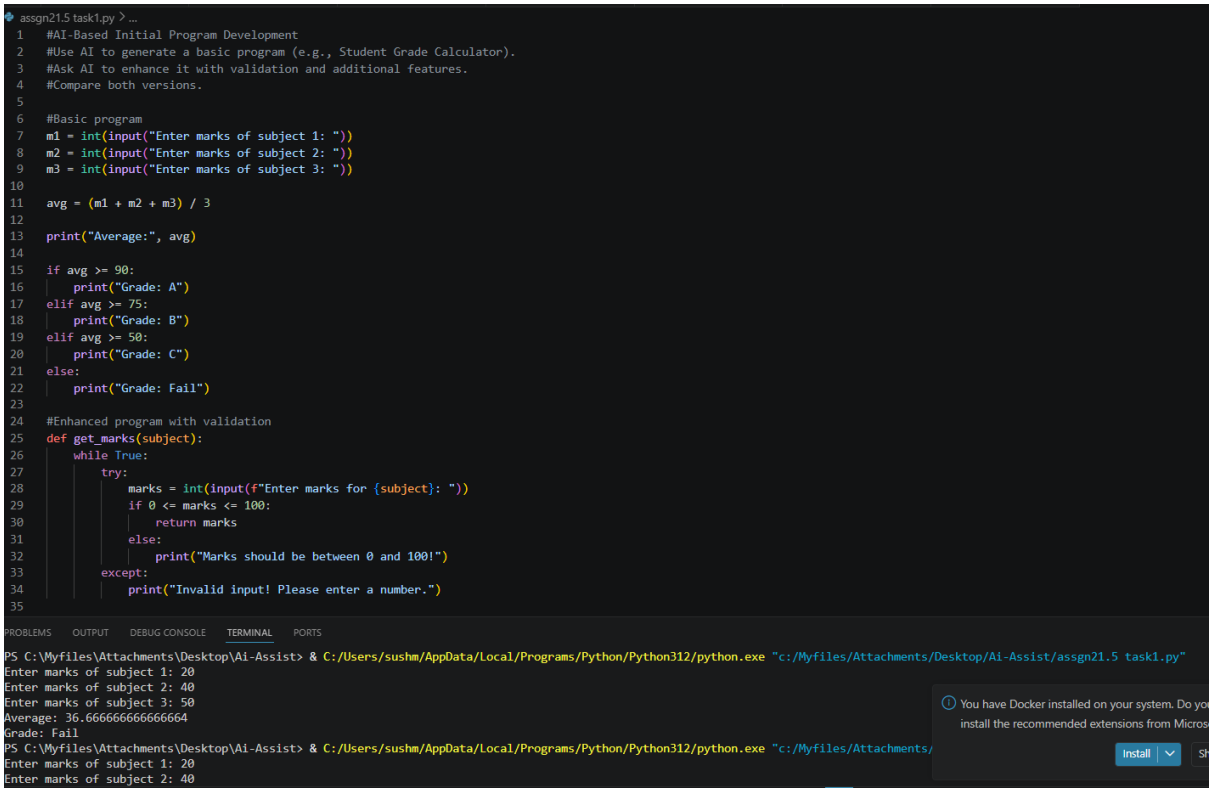
Prompt:

Generate a Python program for a Student Grade Calculator that takes marks of 3 subjects as input and calculates the average and grade.

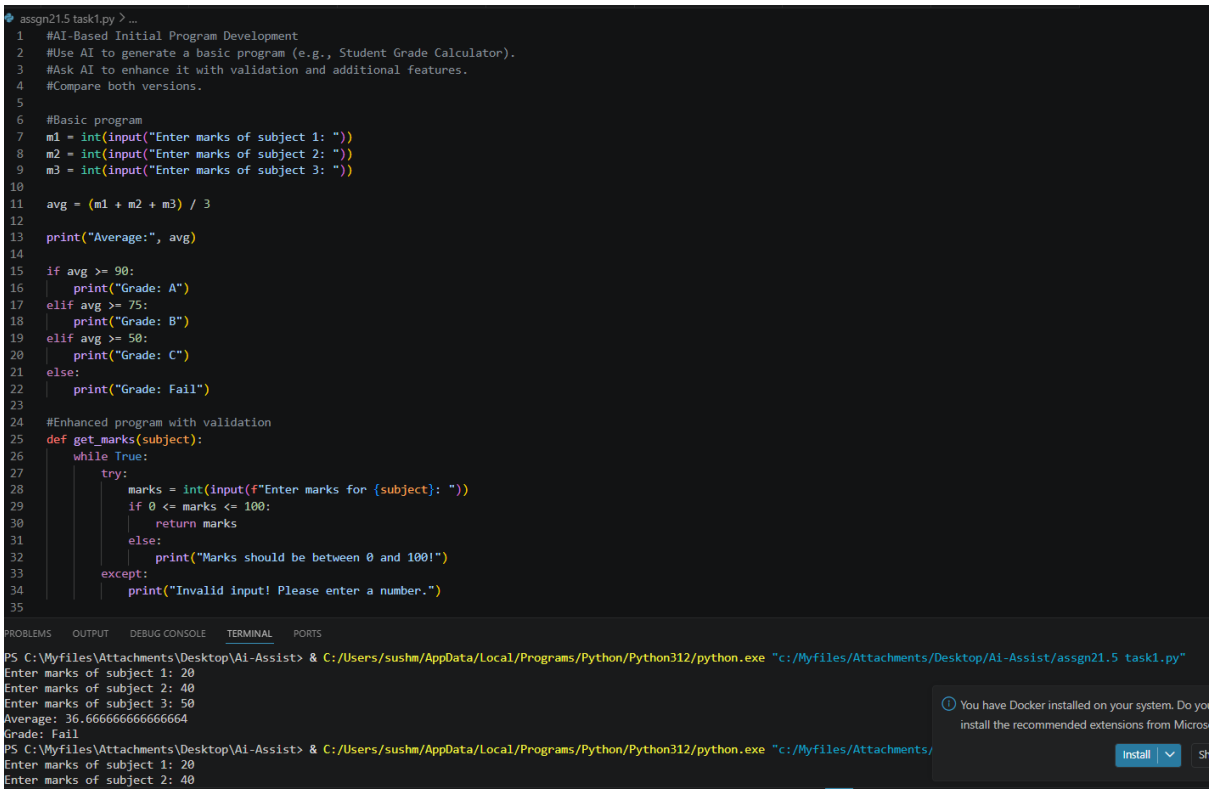
Then enhance the program by adding:

- Input validation (marks between 0–100)
- Exception handling for invalid inputs
- Correct average calculation
- Total marks display
- Pass/Fail status based on individual subject marks
- User-friendly output

Code:



```
assgn21.5 task1.py > ...
#AI-Based Initial Program Development
#Use AI to generate a basic program (e.g., Student Grade Calculator).
#Ask AI to enhance it with validation and additional features.
#Compare both versions.

#Basic program
m1 = int(input("Enter marks of subject 1: "))
m2 = int(input("Enter marks of subject 2: "))
m3 = int(input("Enter marks of subject 3: "))

avg = (m1 + m2 + m3) / 3

print("Average:", avg)

if avg >= 90:
    print("Grade: A")
elif avg >= 75:
    print("Grade: B")
elif avg >= 50:
    print("Grade: C")
else:
    print("Grade: Fail")

#Enhanced program with validation
def get_marks(subject):
    while True:
        try:
            marks = int(input(f"Enter marks for {subject}: "))
            if 0 <= marks <= 100:
                return marks
            else:
                print("Marks should be between 0 and 100!")
        except:
            print("Invalid input! Please enter a number.")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn21.5 task1.py"
Enter marks of subject 1: 20
Enter marks of subject 2: 40
Enter marks of subject 3: 50
Average: 36.666666666666664
Grade: Fail
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/
Enter marks of subject 1: 20
Enter marks of subject 2: 40
```

```
assgn21.5 task1.py > ...
#AI-Based Initial Program Development
#Use AI to generate a basic program (e.g., Student Grade Calculator).
#Ask AI to enhance it with validation and additional features.
#Compare both versions.

#Basic program
m1 = int(input("Enter marks of subject 1: "))
m2 = int(input("Enter marks of subject 2: "))
m3 = int(input("Enter marks of subject 3: "))

avg = (m1 + m2 + m3) / 3

print("Average:", avg)

if avg >= 90:
    print("Grade: A")
elif avg >= 75:
    print("Grade: B")
elif avg >= 50:
    print("Grade: C")
else:
    print("Grade: Fail")

#Enhanced program with validation
def get_marks(subject):
    while True:
        try:
            marks = int(input(f"Enter marks for {subject}: "))
            if 0 <= marks <= 100:
                return marks
            else:
                print("Marks should be between 0 and 100!")
        except:
            print("Invalid input! Please enter a number.")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn21.5 task1.py"
Enter marks of subject 1: 20
Enter marks of subject 2: 40
Enter marks of subject 3: 50
Average: 36.666666666666664
Grade: Fail
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/
Enter marks of subject 1: 20
Enter marks of subject 2: 40
```


Observation:

The initial AI program was simple and worked only for valid inputs but lacked validation and error handling. After refining the prompt, the improved version became more reliable, user-friendly, and

included real-world features. This shows that better prompts lead to better AI-generated code.

Task B – AI-Assisted Application Improvement

Prompt:

Generate a simple Python program for a Library Management system to add and display books.

Then enhance the program by adding:

- Search functionality
- Book issue and return system
- Input validation
- Better structured code using functions

Code:

assgn21.5 task2.py > ...

```python
# AI-Assisted Application Improvement
#Generate a simple program (e.g., Library Management).
#Use AI to add new functionalities.
#Analyze improvements in structure and performance.

#Basic
books = []

n = int(input("Enter number of books: "))

for i in range(n):
    name = input("Enter book name: ")
    books.append(name)

print("\nBooks in Library:")
for b in books:
    print(b)

#Enhanced with AI
books = []
issued = []

def add_book():
    name = input("Enter book name: ")
    books.append(name)
    print("Book added!")

def display_books():
    print("\nAvailable Books:")
    for b in books:
        if b not in issued:
            print(b)

def search_book():
    name = input("Enter book to search: ")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

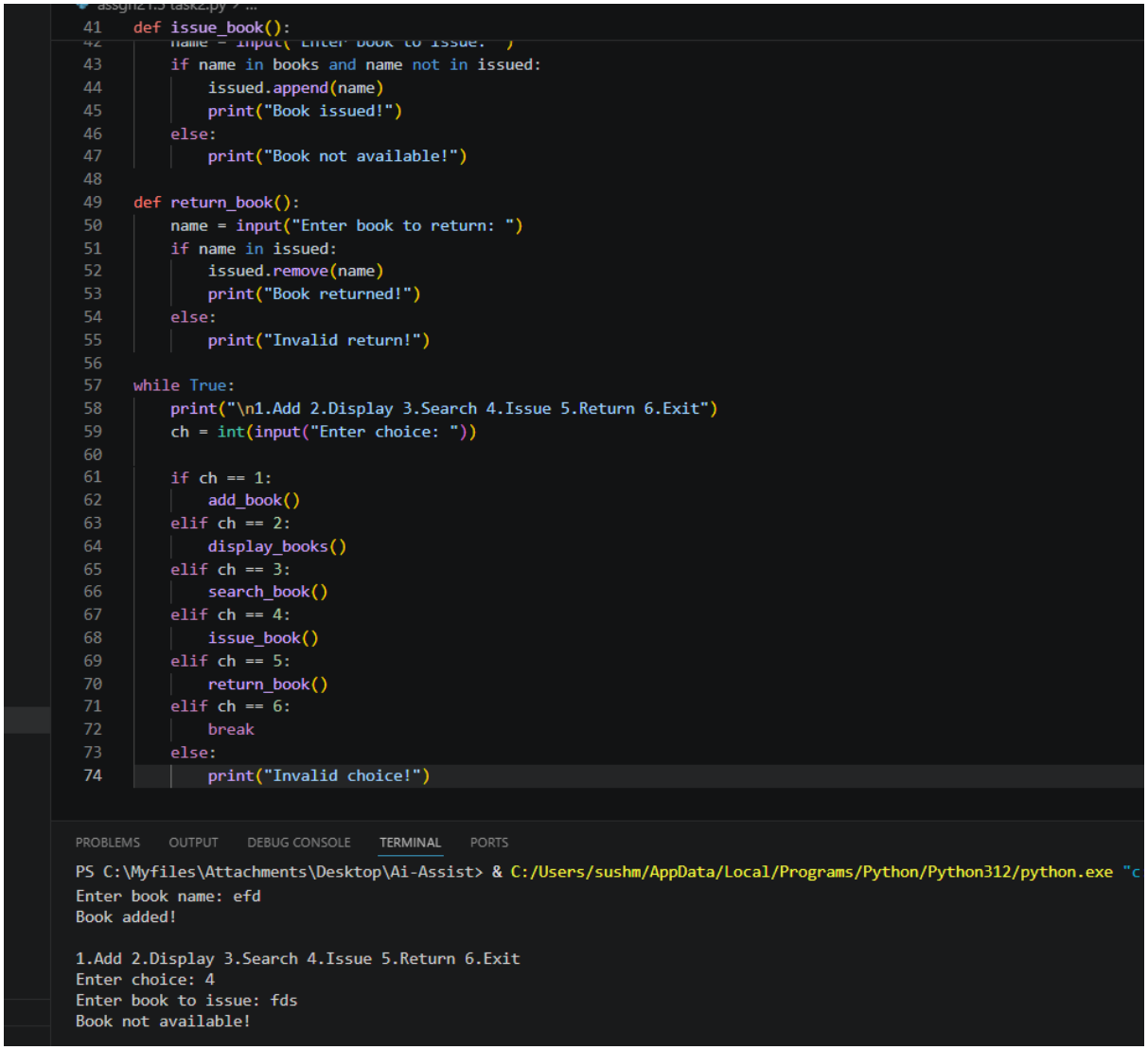
```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments
Enter number of books: 3
Enter book name: abc
Enter book name: dge
Enter book name: bdg

Books in Library:
abc
dge
```

assgn21.5 task2.py > ...

```python
def search_book():
    name = input("Enter book to search: ")
    if name in books:
        print("Book found!")
    else:
        print("Book not found!")

def issue_book():
    name = input("Enter book to issue: ")
    if name in books and name not in issued:
        issued.append(name)
        print("Book issued!")
    else:
        print("Book not available!")

def return_book():
    name = input("Enter book to return: ")
    if name in issued:
        issued.remove(name)
        print("Book returned!")
    else:
        print("Invalid return!")

while True:
    print("\n1.Add 2.Display 3.Search 4.Issue 5.Return 6.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        add_book()
    elif ch == 2:
        display_books()
    elif ch == 3:
        search_book()
    elif ch == 4:
        issue_book()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn21.5 t
Books in Library:
abc
dge
bdg

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Exit
Enter choice: 1
Enter book name: efd
```



```
def issue_book():
    name = input("Enter book to issue: ")
    if name in books and name not in issued:
        issued.append(name)
        print("Book issued!")
    else:
        print("Book not available!")

def return_book():
    name = input("Enter book to return: ")
    if name in issued:
        issued.remove(name)
        print("Book returned!")
    else:
        print("Invalid return!")

while True:
    print("\n1.Add 2.Display 3.Search 4.Issue 5.Return 6.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        add_book()
    elif ch == 2:
        display_books()
    elif ch == 3:
        search_book()
    elif ch == 4:
        issue_book()
    elif ch == 5:
        return_book()
    elif ch == 6:
        break
    else:
        print("Invalid choice!")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c
Enter book name: efd
Book added!

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Exit
Enter choice: 4
Enter book to issue: fds
Book not available!
```

Observation:

- Basic program was simple and limited
- Enhanced version added search, issue, return features
- Code structure improved using functions
- Program became more interactive and user-friendly
- Shows that AI can improve both functionality and structure

Task C – Git Workflow Practice

Prompt:

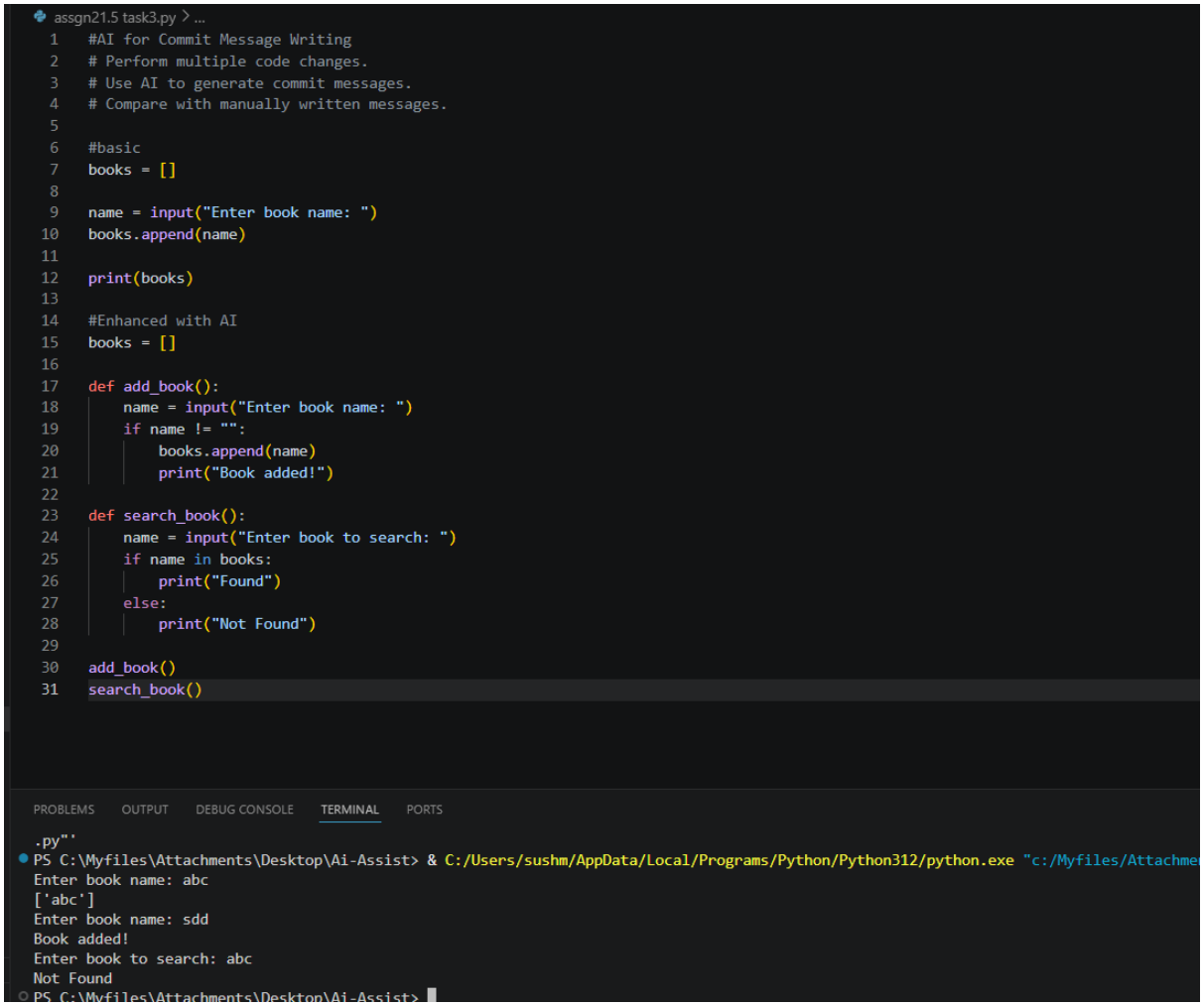
Generate a clear and concise Git commit message for the following changes:

- Added search functionality

- Implemented input validation

- Improved code structure using functions

Code:



Observation:

AI-generated commit messages are more clear, specific, and structured compared to manually written messages. They describe the exact changes made in the code, which improves readability and project tracking. Manual messages are quicker to write but may be

vague or inconsistent. This shows that AI helps in generating better quality commit messages and saves time.

Task D – Pair Programming with AI Assistance

Prompt:

Improve the given Python library management code by adding:

- Input validation
- Search functionality
- Issue and return system
- Better structure using functions
- User-friendly interaction

Code:

assgn21.5 task4.py > ...

```python
#Pair Programming with AI Assistance
# Student A develops initial code.
# Student B uses AI to enhance and optimize.
# Integrate changes using Git.

#basic
books = []

n = int(input("Enter number of books: "))

for i in range(n):
    name = input("Enter book name: ")
    books.append(name)

print("Books:", books)

books = []
issued = []

def add_book():
    name = input("Enter book name: ")
    if name != "":
        books.append(name)
        print("Book added!")

def display_books():
    print("\nAvailable Books:")
    for b in books:
        if b not in issued:
            print(b)

def search_book():
    name = input("Enter book to search: ")
    if name in books:
        print("Book found!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/
Enter number of books: 3
Enter book name: rdf
Enter book name: ee
Enter book name: wws
Books: ['rdf', 'ee', 'wws']

1.Add 2.Display 3.Search 4.Issue 5.Return 6.Exit
Enter choice: w
```

assgn21.5 task4.py > ...

```python
def display_books():
    for b in books:
        if b not in issued:
            print(b)

def search_book():
    name = input("Enter book to search: ")
    if name in books:
        print("Book found!")
    else:
        print("Not found!")

def issue_book():
    name = input("Enter book to issue: ")
    if name in books and name not in issued:
        issued.append(name)
        print("Book issued!")
    else:
        print("Not available!")

def return_book():
    name = input("Enter book to return: ")
    if name in issued:
        issued.remove(name)
        print("Book returned!")
    else:
        print("Invalid!")

while True:
    print("\n1.Add 2.Display 3.Search 4.Issue 5.Return 6.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        add_book()
    elif ch == 2:
        display_books()
    elif ch == 3:
        search_book()
    elif ch == 4:
        issue_book()
    elif ch == 5:
        return_book()
    elif ch == 6:
        break
    else:
        print("Invalid choice!")
```

You have Docker installed on your system. Do
install the recommended extensions from Mic

Install

Observation:

Pair programming with AI improved both code quality and development speed. Student A created a basic version, while Student B used AI to enhance it with better features and structure. The final program became more efficient, user-friendly, and maintainable. Git helped in managing and merging contributions smoothly without affecting the original code.

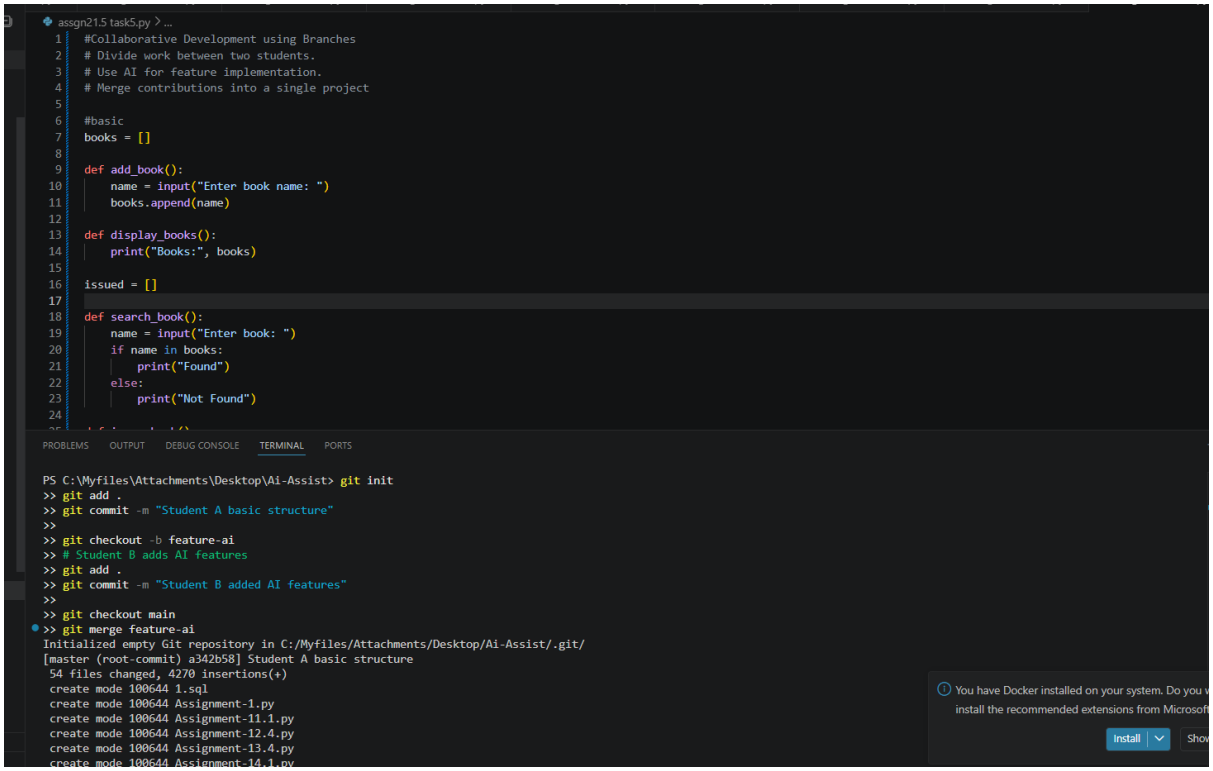
Task E – Collaborative Development using Branches

Prompt:

Generate Python functions for a library system including:

- Search book feature
- Issue and return book functionality
- Input validation
- Menu-driven interaction

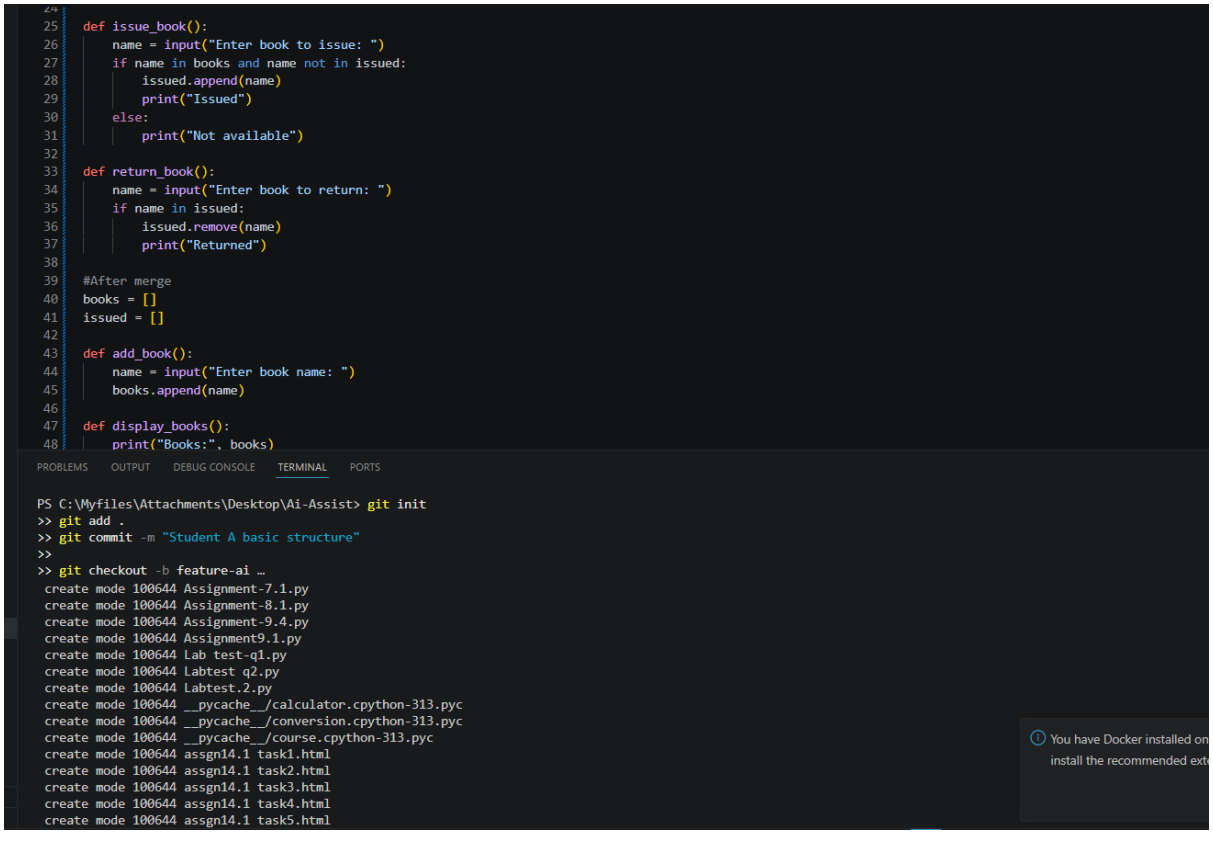
Code:



```
assgn21.5 task5.py > ...
#Collaborative Development using Branches
# Divide work between two students.
# Use AI for feature implementation.
# Merge contributions into a single project

#basic
books = []

def add_book():
    name = input("Enter book name: ")
    books.append(name)

def display_books():
    print("Books:", books)

issued = []

def search_book():
    name = input("Enter book: ")
    if name in books:
        print("Found")
    else:
        print("Not Found")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> git init
>> git add .
>> git commit -m "Student A basic structure"
>>
>> git checkout -b feature-ai
>> # Student B adds AI features
>> git add .
>> git commit -m "Student B added AI features"
>>
>> git checkout main
>> git merge feature-ai
Initialized empty Git repository in C:/Myfiles/Attachments/Desktop/Ai-Assist/.git/
[master (root-commit) a342b58] Student A basic structure
 54 files changed, 4270 insertions(+)
 create mode 100644 1.sql
 create mode 100644 Assignment-1.py
 create mode 100644 Assignment-11.1.py
 create mode 100644 Assignment-12.4.py
 create mode 100644 Assignment-13.4.py
 create mode 100644 Assignment-14.1.py
```

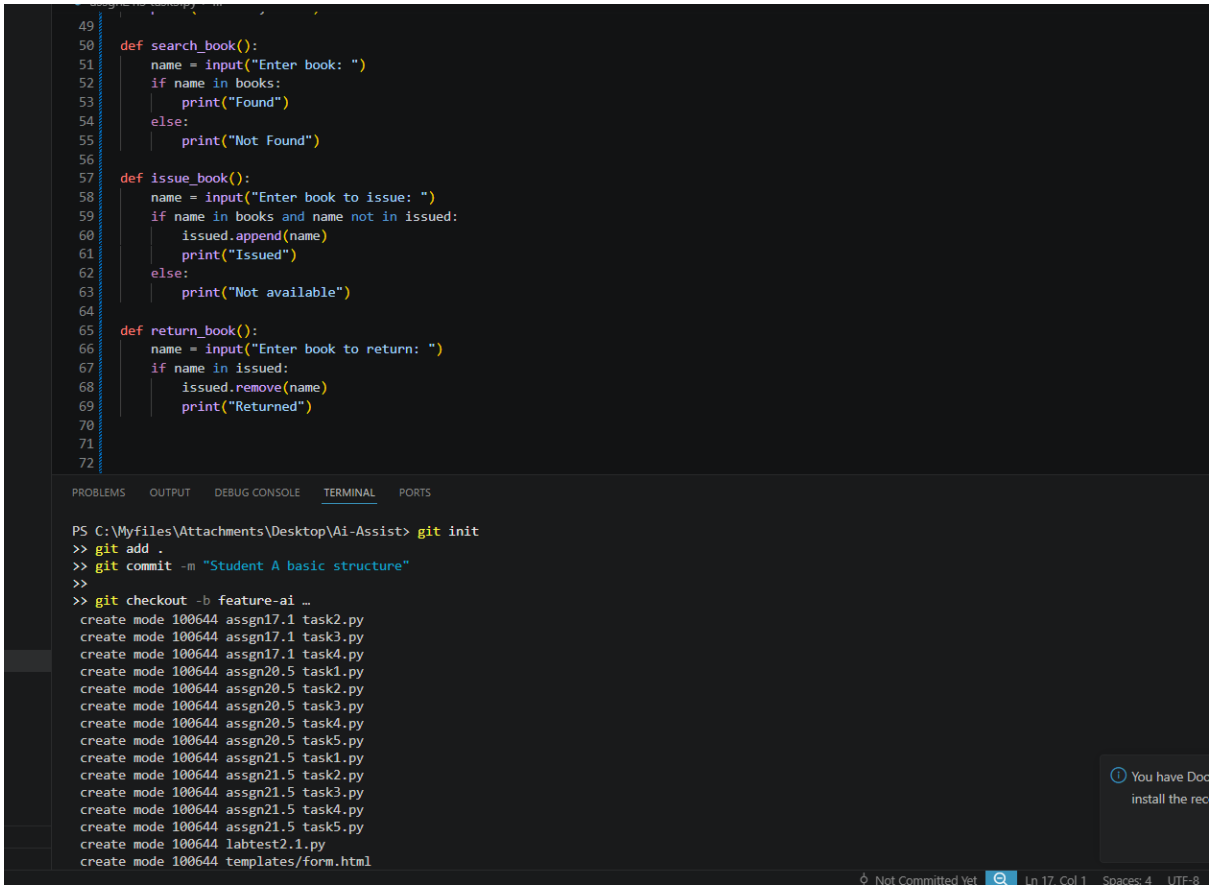
You have Docker installed on your system. Do you w
install the recommended extensions from Microsoft
Install | Show



```
def issue_book():
    name = input("Enter book to issue: ")
    if name in books and name not in issued:
        issued.append(name)
        print("Issued")
    else:
        print("Not available")

def return_book():
    name = input("Enter book to return: ")
    if name in issued:
        issued.remove(name)
        print("Returned")

#After merge
books = []
issued = []

def add_book():
    name = input("Enter book name: ")
    books.append(name)

def display_books():
    print("Books:", books)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> git init
>> git add .
>> git commit -m "Student A basic structure"
>>
>> git checkout -b feature-ai …
 create mode 100644 Assignment-7.1.py
 create mode 100644 Assignment-8.1.py
 create mode 100644 Assignment-9.4.py
 create mode 100644 Assignment9.1.py
 create mode 100644 Lab test-q1.py
 create mode 100644 Labtest q2.py
 create mode 100644 Labtest.2.py
 create mode 100644 __pycache__/calculator.cpython-313.pyc
 create mode 100644 __pycache__/conversion.cpython-313.pyc
 create mode 100644 __pycache__/course.cpython-313.pyc
 create mode 100644 assgn14.1 task1.html
 create mode 100644 assgn14.1 task2.html
 create mode 100644 assgn14.1 task3.html
 create mode 100644 assgn14.1 task4.html
 create mode 100644 assgn14.1 task5.html
```

You have Docker installed on
install the recommended ext



Observation:

Collaborative development using branches allows multiple students to work independently without conflicts. Student A created the base structure, while Student B added AI-based features in a separate branch. After merging, the final project combined all functionalities efficiently. This approach improves teamwork, code organization, and development speed.